RUBY + SINATRA + SQLITE

Uma API pequena. Um fluxo completo.

Gerenciamento de tarefas, do navegador ao banco de dados.

Fernando Buligon

Seminário de TDS · UTFPR

Uma tarefa, quatro operações.

Um exemplo prático para demonstrar o CRUD de uma API.

C

Criar

Cadastrar uma nova tarefa.

R

Ler

Consultar a lista de tarefas.

U

Atualizar

Editar o título ou a conclusão.

D

Excluir

Remover uma tarefa da lista.

Uma rota liga HTTP a um bloco Ruby.

Sinatra é um framework web para Ruby.

A aplicação define o que executar para cada método e caminho da requisição.

```ruby
get '/tasks' do
  all_tasks.to_json
end
```

> **GET** escolhe a operação. **/tasks** identifica o recurso. O bloco produz a resposta.

Cada tecnologia tem um papel.

Ruby + Sinatra	Rotas, validações e regras da API.
Puma + Rack	Servidor HTTP e integração com a aplicação.
SQLite	Persistência em arquivo local, via SQL.
HTML + CSS + JavaScript	Interface que consome a API com fetch.

Do clique ao dado salvo.

01

Navegador

Interface + fetch

`public/app.js`

→

02

API Sinatra

Rotas + validações

`app.rb`

→

03

SQLite

Consultas + persistência

`tasks.sqlite3`

> Na volta: o banco retorna dados, a API responde em JSON e a interface atualiza a lista.

Três campos descrevem uma tarefa.

JSON · exemplo

```
{
  "id": 1,
  "title": "Preparar seminário",
  "completed": false
}
```

id

Inteiro gerado pelo banco.

title

Texto obrigatório e não vazio.

completed

Booleano: pendente ou concluída.

No SQLite, completed é armazenado como 0 ou 1; na API, vira false ou true.

O contrato da API.

O método indica a ação; a URL identifica as tarefas.

MÉTODO	ROTA	AÇÃO	HTTP
GET	`/tasks`	Listar	200
POST	`/tasks`	Criar	201
PATCH	`/tasks/:id`	Atualizar	200
DELETE	`/tasks/:id`	Excluir	204

PATCH preserva os campos omitidos. DELETE retorna sucesso sem corpo.

Criar: uma requisição, uma resposta.

POST /tasks · application/json

```
{
  "title": "Preparar seminário"
}
```

201 Created · application/json

```
{
  "id": 1,
  "title": "Preparar seminário",
  "completed": false
}
```

> O cliente envia o título. O servidor devolve também o ID e o estado inicial da tarefa.

O servidor reinicia. As tarefas continuam.

ARQUIVO DO BANCO `data/tasks.sqlite3`

01

Salvar

Cada alteração é gravada no SQLite.

02

Reiniciar

Encerrar o Ruby não apaga o arquivo.

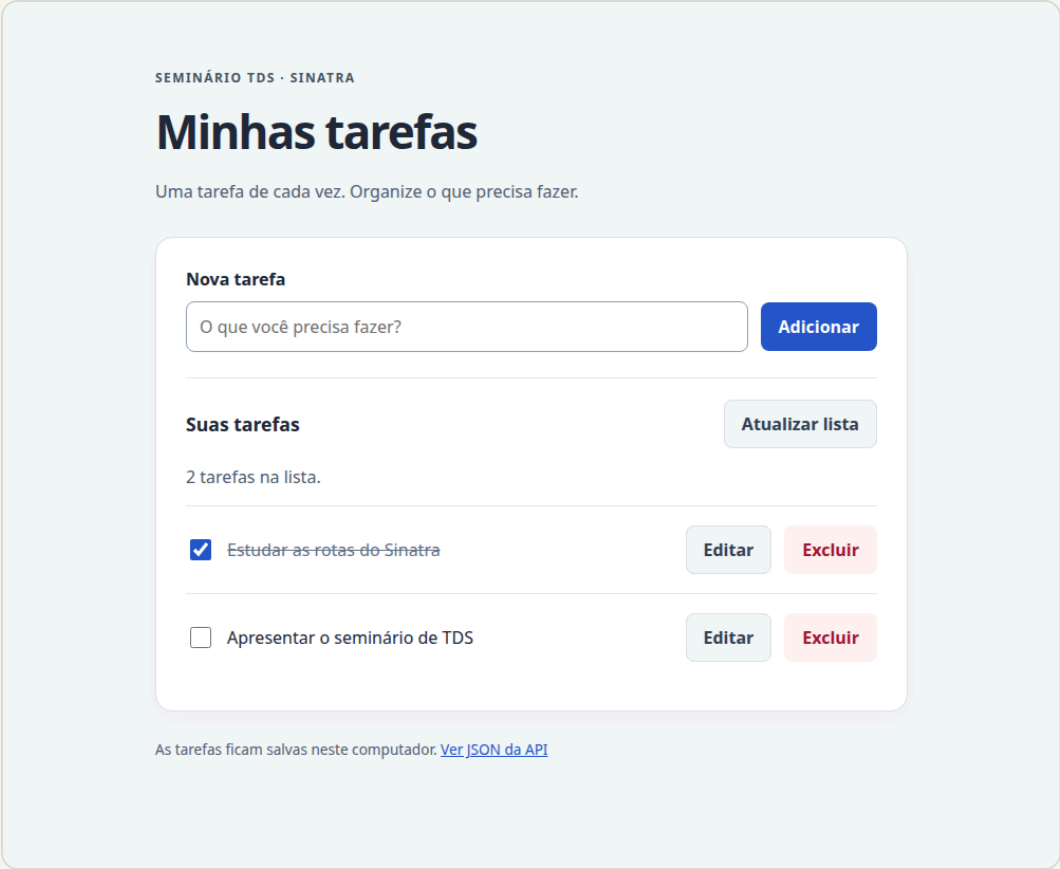
03

Consultar

A API lê as tarefas já armazenadas.

O diretório, o banco e a tabela são criados automaticamente na primeira execução.

A API ganha uma interface.



Interface real do projeto com tarefas ilustrativas. Acesso: http://127.0.0.1:4567.

Entradas inválidas. Respostas claras.

Título

String não vazia, com espaços externos removidos.

Conclusão

Apenas true ou false, sem aspas.

Identificador

Inteiro positivo; deve corresponder a uma tarefa.

404 · exemplo de erro

```
{
  "error": "Recurso não encontrado."
}
```

400: entrada inválida.

404: recurso ou rota inexistente.

Vamos ver funcionando.

01 **Iniciar o servidor**

```
bundle exec ruby app.rb
```

02 **Abrir a interface**

http://127.0.0.1:4567 ↗

03 **Criar → editar → concluir**

Mostrar também o JSON em /tasks.

04 **Reiniciar → conferir → excluir**

A tarefa deve continuar salva.

O QUE O PROJETO DEMONSTRA

Rotas claras.
Dados persistentes.
Interface simples.

github.com/FernandoBuligon/seminario-tds ↗